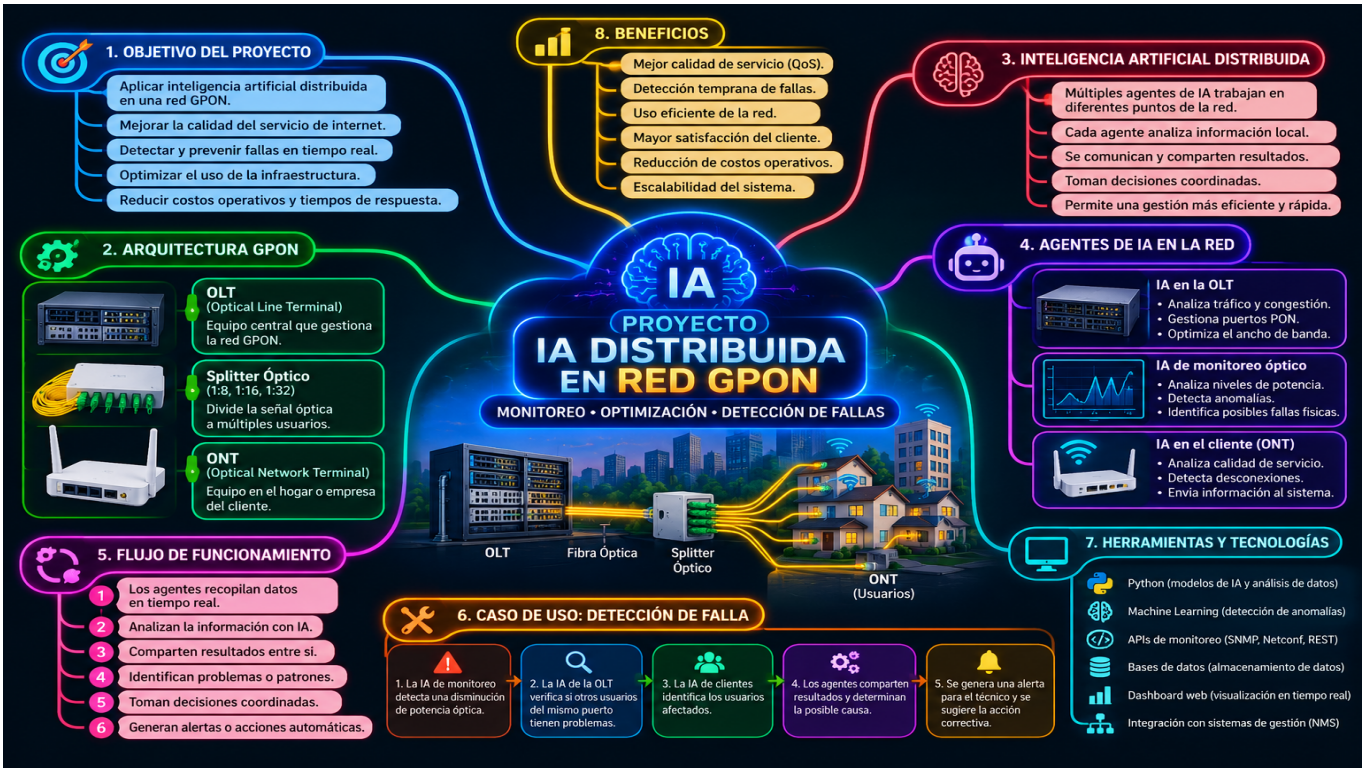
JONGOX SMART CITY 3D

GUÍA MAESTRA PARA CODEX

Gemelo digital urbano 3D con inteligencia artificial distribuida, red eléctrica y telecomunicaciones GPON.



Documento de trabajo · Octubre de 2026

Uso: Guarda este PDF dentro de la carpeta JONGOX SMART CITY 3D. Abre esa carpeta en VS Code y proporciona a Codex este documento o copia los prompts por etapa. El PDF es una especificación: Codex deberá crear, ejecutar y probar el código.

Alcance: ciudad 3D realista optimizada para PC básico, cuatro zonas, 5.000 abonados GPON lógicos, red eléctrica, alumbrado, semáforos, dashboard y agentes de IA distribuidos.

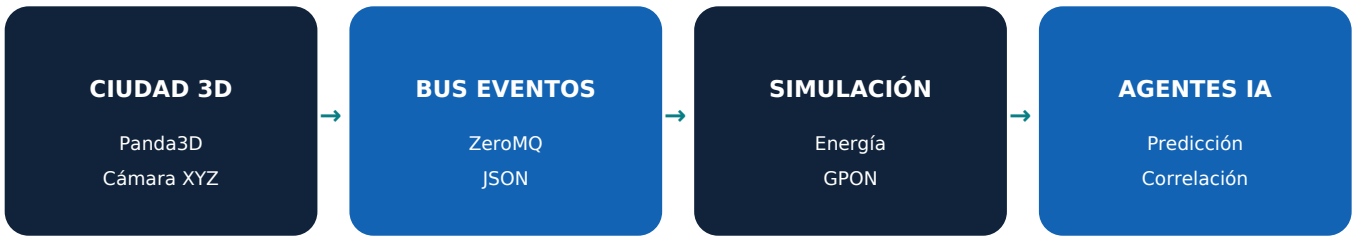
1. Visión y reglas de ingeniería

Objetivo académico: demostrar cooperación entre agentes inteligentes especializados. La red GPON por sí sola no es IA distribuida; los agentes independientes analizan telemetría, comparten eventos y correlacionan incidentes.

Restricciones: Windows, Python 3.11, 4–8 GB de RAM como objetivo de optimización, gráficos integrados, operación sin internet después de instalar dependencias. Las métricas de rendimiento son metas que deberán medirse, no promesas.

Elemento	Decisión
Motor 3D	Panda3D; geometría procedural, instanciación y niveles de detalle
Dashboard	PySide6; proceso separado
IA	scikit-learn: IsolationForest y pronóstico supervisado ligero
Mensajería	ZeroMQ; eventos JSON versionados y canal fiable de comandos
Persistencia	SQLite WAL; un escritor, agregación y retención
Pruebas	pytest, simulaciones reproducibles y benchmark

2. Arquitectura general



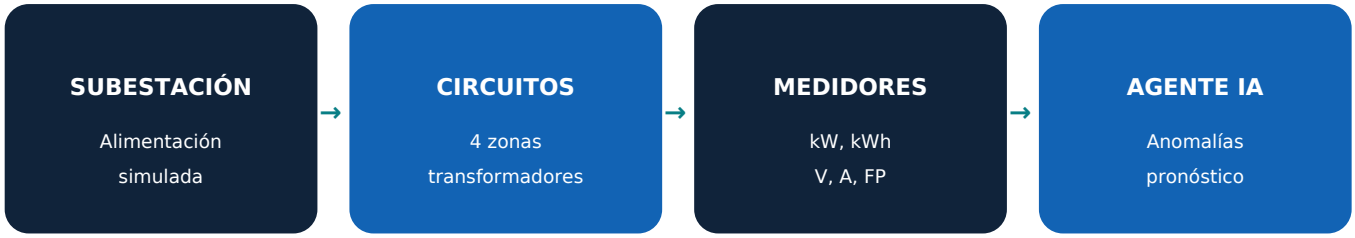
El dashboard escucha los mismos eventos y consulta datos agregados. Un lanzador main.py supervisa los procesos. La simulación debe continuar si la ventana 3D se cierra, y reiniciar la visualización no debe duplicar inventarios.

3. División de la metrópolis

Zona	Perfil	Abonados GPON
A	Residencial: viviendas, medidores, luminarias	2.200
B	Comercial: locales, edificios y cruces	1.400
C	Industrial: fábricas, cargas y transformadores	600
D	Tecnológica: hospital, NOC, OLT y centro de datos	800
TOTAL	4 zonas, registros lógicos, edificios 3D representativos	5.000

No dibujar 5.000 casas: cada edificio 3D representa varios usuarios en la base de datos. Cada usuario mantiene su ONT, medidor, circuito, puerto PON, zona y dependencias eléctricas.

4. Red eléctrica inteligente



Modelo académico radial trifásico simplificado: potencia activa kW, energía acumulada kWh, tensión V, corriente A, factor de potencia, carga de transformador y frecuencia. Integrar energía mediante kWh += kW × delta_horas. Simular perfiles horarios con semilla fija.

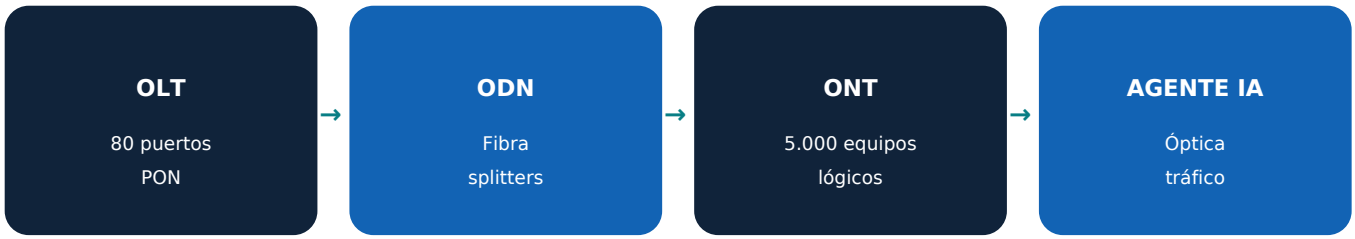
```
energy_kwh += power_kw * elapsed_seconds / 3600.0

loading_pct = 100.0 * apparent_kva / rated_kva

# El modelo debe distinguir potencia (kW) y energía (kWh).
```

Casos: caída de alimentador, sobrecarga de transformador, caída de tensión, alumbrado fuera de horario, semáforo sin alimentación. La restauración debe recuperar estados dependientes. Las operaciones de control serán exclusivamente simuladas.

5. Telecomunicaciones GPON



Topología propuesta: 80 puertos PON con máximo nominal de 64 ONT por puerto, capacidad teórica de 5.120 ONT. Asignar 5.000 sin superar 64 por puerto. Separar GPON de XGS-PON y parametrizar clase óptica, presupuesto, capacidad de enlace y límites de alarma.

```
rx_dbm = tx_dbm - (fiber_km * loss_db_per_km) \

        - splitter_loss_db - connectors_loss_db - splice_loss_db

# Comparar Rx contra umbrales configurados por clase óptica.
```

Monitorear Tx/Rx dBm, longitud, atenuación, LOS, registro ONT, tráfico ascendente/descendente, utilización PON, latencia, pérdida de paquetes y alimentación eléctrica. Considerar UPS opcional y correlación con cortes eléctricos.

6. Inteligencia artificial distribuida

Agente	Datos	Resultado
EnergyAgent	kW, V, carga, FP	Anomalía eléctrica y evidencia
LoadForecastAgent	Consumo histórico por zona	Pronóstico y error MAE/RMSE
GPONAgent	Rx, LOS, tráfico, estado	Anomalía óptica o conectividad
NetworkForecastAgent	Utilización histórica PON	Pronóstico de congestión
UrbanAgent	Semáforos y alumbrado	Incidentes urbanos
CorrelationAgent	Eventos + dependencias	Hipótesis y confianza

Cada agente es independiente: recibe eventos, mantiene estado, emite diagnósticos, heartbeat y registros. Usar reglas de respaldo si faltan modelos. Los datos son sintéticos y los diagnósticos probabilísticos; no atribuir certeza a una simple coincidencia temporal.

Entrenamiento: generar escenarios normales y fallas etiquetadas; evitar fuga temporal; dividir entrenamiento/prueba; evaluar precision, recall y F1 en detección cuando existan etiquetas, y MAE/RMSE en pronósticos. Guardar versión, variables y métricas de cada modelo.

7. Contrato de eventos y dependencias

```
{
 "schema_version": 1,
 "event_id": "uuid",
 "timestamp_utc": "2026-10-20T19:00:00Z",
 "simulation_time": 3600,
 "source": "EnergyAgent",
 "zone_id": "A",
 "asset_id": "TX-A-01",
 "event_type": "transformer_outage",
 "severity": "critical",
 "correlation_id": "uuid",
 "payload": {"affected_circuits": ["A-01"]}
}
```

Implementar eventos idempotentes, esquema versionado, validación, reconexión y manejo de mensajes tardíos. La base de dependencias debe relacionar transformador → circuito → medidor/ONT y OLT → puerto PON → splitter → ONT.

8. Escenario principal de demostración

Falla TX-A-01: se corta un transformador. EnergyAgent detecta interrupción; ONT sin UPS pierden alimentación; GPONAgent reporta desconexiones; CorrelationAgent identifica una causa común basada en topología, zona y tiempo. El dashboard muestra causa probable, equipos afectados, evidencia y restauración.

9. Organización del repositorio

```
JONGOX SMART CITY 3D/
  main.py
  requirements.txt
  README.md
  .env.example
  config/settings.py
  config/city_config.json
  city3d/{app,camera_controller,city_generator}.py
  city3d/{buildings,roads,equipment,network_layers}.py
  simulation/{engine,electrical_grid,smart_meters}.py
  simulation/{gpon_network,ont_devices,fault_injector}.py
  simulation/{traffic_lights,street_lighting}.py
  agents/{base_agent,energy_agent,gpon_agent}.py
  agents/{load_forecast_agent,network_forecast_agent}.py
  agents/{urban_agent,correlation_agent}.py
  ai/{features,training,forecasting,model_registry}.py
  communications/{broker,publisher,subscriber,schemas}.py
  dashboard/{app,main_window,overview,alarm_panel}.py
  dashboard/{electrical_panel,gpon_panel,charts}.py
  database/{connection,models,repository,migrations}.py
  assets/{models,textures,icons}/
  scripts/{seed_city,train_models,benchmark}.py
  tests/{test_electrical,test_gpon,test_agents}.py
  tests/{test_events,test_faults,test_integration}.py
```

10. Clases y responsabilidades

Clase	Responsabilidad
SmartCityApp / CameraController	Renderizado, cámara orbital XYZ y selección
CityGenerator	Generación procedural de cuatro zonas
SimulationEngine	Reloj, ciclos y semilla reproducible
ElectricalGrid / SmartMeter	Circuitos, transformadores y consumos
GPONNetwork / ONTDevice	Inventario y telemetría GPON
BaseAgent / CorrelationAgent	Suscripción, inferencia y cooperación
FaultInjector	Inyección, registro y restauración
MonitoringDashboard	Paneles, alarmas, tendencias y comandos

11. Dependencias e instalación

```
cd "JONGOX SMART CITY 3D"

py -3.11 -m venv .venv

.venv\Scripts\activate

python -m pip install --upgrade pip

pip install panda3d PySide6 numpy pandas scikit-learn pyzmq \

 matplotlib joblib pydantic pytest

python main.py
```

Codex debe generar requirements.txt con versiones compatibles y probadas. SQLite forma parte de la biblioteca estándar. El lanzador debe informar errores de dependencias y apagarse limpiamente.

12. PROMPT MAESTRO: instrucción inicial para Codex

Copia esta instrucción primero. Después ejecuta las etapas siguientes en el orden indicado.

ROL: Actúa como arquitecto senior de Python, Panda3D, GPON, redes eléctricas de distribución y sistemas multiagente. Trabaja dentro de la carpeta actual JONGOX SMART CITY 3D. Implementa archivos reales y verificables, no pseudocódigo. Lee esta guía como especificación de referencia. El proyecto es un gemelo digital académico offline; no controla infraestructura real.

RESULTADO: entregar dos aplicaciones visibles: ciudad 3D y centro de monitoreo; cuatro zonas; 5.000 ONT lógicas; energía, GPON, alumbrado, semáforos; seis agentes distribuidos; inyección de fallas; modelos predictivos evaluados; pruebas y documentación.

REGLAS: prioriza funcionalidad y rendimiento en 4–8 GB RAM; geometría procedural sin descargas externas; no renderizar 5.000 casas; desacoplar procesos; un solo escritor SQLite; mensajes versionados; datos sintéticos reproducibles; no afirmar resultados no ejecutados; sin botones falsos; controles eléctricos únicamente simulados.

ENTREGA: construye por etapas internas, valida cada módulo y al final entrega un programa integral iniciable con python main.py. Si el contexto es insuficiente, completa primero un núcleo mínimo real y documenta los pendientes sin ocultarlos.

13. PROMPTS POR ETAPA PARA CODEX

ETAPA 1 · Base, configuración y datos

Instrucción para Codex: Crea el repositorio modular, requirements.txt, README, settings, SQLite WAL y generador de inventario con semilla fija. Genera 5.000 ONT únicas en cuatro zonas (2200/1400/600/800), 80 puertos PON y máximo 64 ONT por puerto. Incluye tests que validen cantidad, unicidad, capacidad, claves y relaciones. Ejecuta pytest y reporta resultados.

ETAPA 2 · Ciudad 3D

Instrucción para Codex: Implementa Panda3D con terreno, cuatro barrios diferenciados, edificios instanciados, calles, OLT, gabinetes, transformadores, luminarias y semáforos. Cámara orbital con mouse derecho, desplazamiento con mouse central, zoom rueda, selección de activos y reset R. Capas GPON/eléctrica y modo LOW. No dependas de modelos descargados. Verifica arranque y cierre.

ETAPA 3 · Simulación física

Instrucción para Codex: Implementa reloj reproducible, red eléctrica radial simplificada, medidores, consumo kW/kWh, tensión, transformadores y alumbrado. Implementa topología GPON y telemetría dBm, pérdidas, tráfico, utilización y estados ONT. Crea fallas restaurables y tests de unidades, dependencias y límites de capacidad.

14. PROMPTS POR ETAPA (continuación)

ETAPA 4 · Comunicaciones y persistencia

Instrucción para Codex: Implementa broker ZeroMQ, eventos JSON versionados, validación con Pydantic, heartbeats, reconexión y canal fiable para comandos. Crea un escritor SQLite y agregación de telemetría para evitar crecimiento ilimitado. Prueba duplicados, reinicios y mensajes inválidos.

ETAPA 5 · Inteligencia artificial distribuida

Instrucción para Codex: Implementa seis agentes independientes: EnergyAgent, LoadForecastAgent, GPONAgent, NetworkForecastAgent, UrbanAgent y CorrelationAgent. Usa IsolationForest, modelos ligeros de pronóstico y reglas de respaldo. Entrena offline con datos sintéticos, separación temporal y métricas reales. Implementa grafo de dependencias y explicaciones de correlación. Ejecuta pruebas unitarias.

ETAPA 6 · Centro profesional de monitoreo

Instrucción para Codex: Crea dashboard PySide6 oscuro con indicadores por zona, medidores, OLT/PON/ONT, gráficas, alarmas, búsqueda de activos, diagnósticos de IA y botones funcionales para provocar/restaurar fallas. Actualiza a frecuencia moderada sin bloquear la interfaz. Muestra estados de conexión de los servicios.

ETAPA 7 · Integración integral

Instrucción para Codex: Crea main.py que lance ciudad, simulación, bus, agentes y dashboard, supervise procesos y cierre limpiamente. Sincroniza selección de activos y alertas entre ventanas. Demuestra fallo de transformador con ONT afectadas y correlación; corte de fibra; saturación PON; anomalía eléctrica y fallo urbano. Ejecuta pytest e informa lo verificado.

ETAPA 8 · Rendimiento, documentación y entrega

Instrucción para Codex: Añade benchmark de FPS, memoria y latencia; optimiza gráficos LOW para PC básico; revisa persistencia, errores y desconexiones. Documenta instalación Windows, ejecución, arquitectura, modelos, limitaciones y pruebas. Entrega reporte con comandos exactos, resultados medidos y pendientes, si los hay.

15. Criterios de aceptación

- ☐ main.py abre ciudad 3D y dashboard sin errores críticos.
- ☐ Cuatro zonas navegables, selección y capas funcionales.
- ☐ 5.000 abonados lógicos únicos sin superar 64 ONT por puerto.
- ☐ Medición eléctrica con kW, kWh y unidades coherentes.
- ☐ GPON con inventario, Rx dBm, tráfico, LOS y estados.
- ☐ Seis agentes independientes intercambian eventos y generan resultados.
- ☐ La falla de transformador produce alarmas GPON correlacionadas.
- ☐ Modelos entrenados y evaluados con métricas calculadas.
- ☐ Pruebas pytest ejecutadas y resultados registrados.
- ☐ Modo LOW, offline y apagado limpio verificados.

16. Plan de pruebas y evidencia

ID	Prueba	Criterio
T01	Inventario	5.000 ONT, 80 PON, máximo 64/PON
T02	Energía	kWh coherente con potencia e intervalo
T03	Óptica	Rx = Tx - pérdidas de la ruta
T04	Corte eléctrico	Solo activos dependientes sin respaldo caen
T05	Corte de fibra	Afecta únicamente rama aguas abajo
T06	IA	Métricas reales sobre datos de prueba
T07	Correlación	Causa probable y evidencias trazables
T08	Bus	Reconexión, idempotencia y mensajes inválidos
T09	UI	Mouse XYZ, capas, botones y paneles
T10	Recursos	FPS, RAM y latencia medidos en equipo objetivo

17. Protocolo de demostración ante el profesor

1. Abrir la ciudad y mostrar las cuatro zonas. 2. Abrir el NOC y visualizar 5.000 ONT lógicas. 3. Seleccionar un transformador y mostrar su carga. 4. Inyectar falla de transformador. 5. Mostrar desconexiones GPON y eventos independientes. 6. Mostrar correlación y evidencias de IA. 7. Restaurar servicio y mostrar recuperación. 8. Mostrar métricas de modelos y límites de simulación.

18. Riesgos y límites del proyecto

Una versión integral de este tamaño puede requerir varias iteraciones de Codex. Los primeros resultados pueden incluir módulos incompletos o incompatibilidades: no considerarlos terminados hasta ejecutar pruebas. La geometría realista deberá equilibrarse con la memoria disponible. Los pronósticos sintéticos demuestran la técnica, no validan desempeño en una red real. GPON, red eléctrica y semáforos se modelan con simplificaciones académicas, nunca como sistemas de operación real.

Prioridad de entrega: núcleo funcional y comprobable → fallas reproducibles → agentes cooperativos → interfaz profesional → mejoras visuales. Esta secuencia reduce riesgos sin cambiar el alcance final.

FIN DE LA GUÍA · JONGOX SMART CITY 3D